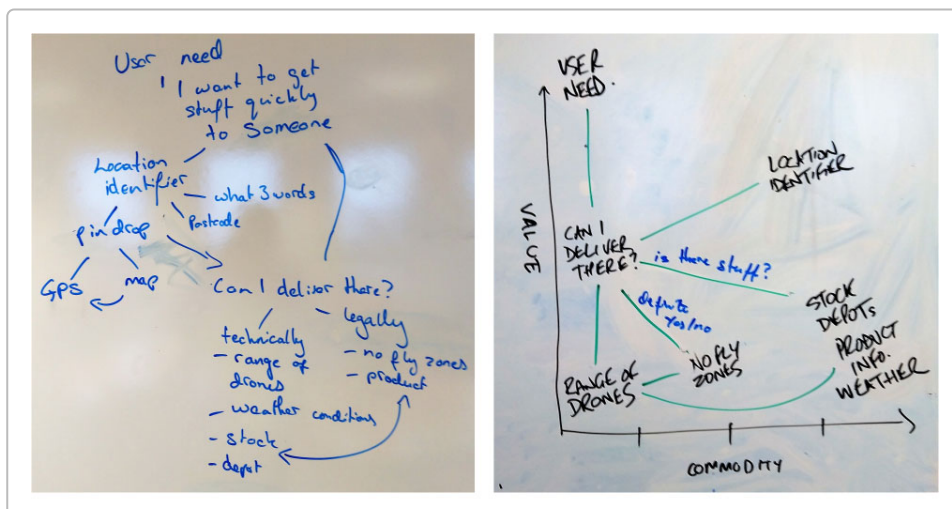


Wardley Maps as an Analogy for Software Development Evolution

Introduction

Software systems are not static – they evolve through distinct stages from experimental beginnings to stable, commoditized services. This evolution mirrors the concept of **Wardley Maps** (developed by Simon Wardley for business strategy), which visualize how components progress from novel ideas to ubiquitous utilities ¹. In this document, we explore how Wardley Maps' evolution stages provide a powerful analogy for software development, and how leveraging this perspective (along with modern tools like LLMs) can improve the way we design, refactor, and reuse code.

Wardley Maps in a Nutshell



An example Wardley map. Components of a drone delivery service are plotted by their value chain position (vertical axis) and their stage of evolution (horizontal axis, from Genesis on the left to Commodity on the right).

A Wardley Map is essentially a **visual map of systems**: it plots a value chain of components needed to satisfy user needs, and positions each component on an evolution axis from *Genesis* to *Commodity* ¹ ². The key idea is that **every component evolves** through four stages – **Genesis, Custom-Built, Product, Commodity** – driven by market pressure and maturity ³ ⁴. Different parts of a system can be at different stages simultaneously ⁴, and understanding this helps in making strategic technical decisions. Below is a brief overview of these stages in Wardley Mapping:

- **Genesis**: A new, novel idea or technology in its earliest form. Solutions in this stage are experimental, **uncertain, and constantly changing** ⁵. There's high risk but also high potential, as you're trying to figure out what the technology or component *could* be.

- **Custom-Built:** A stage where the solution is becoming more defined but is still bespoke. The component is **built in-house, relatively scarce, and evolving** as understanding grows ⁶. There may be only a few implementations worldwide.
- **Product:** A mature, well-understood solution available off-the-shelf. It's **stable, with well-defined interfaces, and widely available** (often from multiple providers) ⁷. Competition exists and best practices are established.
- **Commodity (Utility):** A fully standardized, ubiquitous part of the landscape. It's treated like a utility – **highly reliable, low-cost, and undifferentiated** in the market ⁸. Everyone takes its availability for granted (e.g. open-source libraries, cloud services).

In Wardley Maps, components are connected by dependency lines and anchored on **user needs** ⁹. A critical principle is that **everything on the map should exist for a user-driven reason**. In other words, every component (even low-level ones) should directly or indirectly serve a user need above it ⁹. This will be a useful guiding principle when we translate Wardley mapping to software refactoring decisions.

Software Modules at Different Evolution Stages

In a large software project, you'll typically find modules or components at **various evolutionary stages** concurrently. Some parts of your code are essentially in "*Genesis*" – a freshly written prototype or a hacky script experimenting with new functionality. Others might be "*Custom-Built*" – a unique solution crafted for your product that works but is not yet generic or fully stable. You may also have components that feel like a solid "*Product*" – for example, a well-refactored internal library or service that has a stable API and rarely needs internal changes. Finally, you likely rely on "*Commodity*" elements – standard utilities such as open-source frameworks, database engines, or cloud services that you use as-is because they're proven and ubiquitous.

It's valuable to **identify which stage each part of your system is in**, because it informs how you should treat it. As one author explains, any complex system can be seen as a collection of parts, some in the genesis stage and others more commoditized ². By mapping your software components this way, you gain insight into where you are doing pioneering work versus using well-established solutions. This kind of visibility helps you avoid missteps like treating an experimental subsystem as if it were stable, or reinventing something that's already a commodity.

Focusing Development Effort for Maximum Impact

The Wardley mapping perspective encourages developers to **focus their effort where it yields the most value**. In business terms, Wardley advises investing in components that are in Genesis or Custom-Built stages (where you can differentiate or gain competitive advantage) and **leveraging existing products or utilities for everything else** ¹⁰ ¹¹. For software development, this translates to a few concrete strategies:

- **Invest in evolving the unique parts of your system.** If you have a piece of code that started as a rough prototype but is critical to your product, that is a good candidate for further development and refactoring. By maturing this "custom-built" code toward a product grade, you improve its reliability and create a reusable asset. Each refactoring or redesign is an investment to move that component to a higher stage of evolution where it becomes more stable and efficient. In practice, this might mean turning a script into a proper module, then into a service or library that others can use. The more you **refine and modularize high-value code**, the more effective and robust your overall system becomes. Over time, you accumulate a toolkit of proven components.

- **Avoid reinventing the wheel for commodities.** For needs that are not unique to your business (commodity space), using existing solutions is usually the best move. For example, if you require a logging framework or an authentication service, it's wise to use an established library or cloud service instead of writing your own from scratch. This lets you benefit from others' work and concentrate your limited development time on what truly differentiates your software. As Wardley's mapping insights note, components in the Product/Commodity stage can often be **outsourced or purchased cost-effectively** ¹¹ – in coding terms, that means using open-source, third-party APIs, or services for standard tasks.

A major payoff of pushing components along the evolution spectrum is **increased development speed**. When more of your system is composed of stable, well-understood building blocks, you can assemble new features faster. Simon Wardley observed this effect of commoditization, noting that as components industrialize, it leads not only to efficiency but also *“acceleration in the speed at which I can develop new things.”* ¹² In other words, turning a custom solution into a commodity (e.g. an internal service or published library) doesn't just save you resources on that component – it also enables higher-level innovation at a quicker pace, because you can now build on top of a reliable foundation. This is why, somewhat counter-intuitively, **having more code (in the form of well-factored reusable modules) can make you faster**, not slower. Each mature module is one less thing you have to reinvent next time.

Refactoring with a Purpose: Ensure Every Module Has a “Client”

While improving and refactoring code is beneficial, it must be done **pragmatically and with real needs in mind**. A guiding rule is: **never refactor or abstract a component without a clear client or use-case that needs it**. In Wardley Mapping terms, make sure there is a user need (or a higher-level component) pulling for that evolution ⁹. If no one *above* is going to consume this module, investing effort to polish it might be wasted. This principle is akin to the software maxim YAGNI – “You Aren't Gonna Need It” – which says *don't implement something until it is necessary* ¹³. Refactoring just for the sake of theoretical purity can lead you down rabbit holes (or “tangents”) that have little practical return.

Instead, treat each potential refactor or rewrite as a mini product development effort: **identify the “customer” of that refactoring**. The customer could be an external user, another team, or simply another part of your own system that would benefit from a cleaner utility. For example, you might notice that two or three different features in your application are performing a similar kind of data parsing with slight differences. This is a sign that a common module could be extracted and evolved toward a product-like component – but crucially, you have multiple clients (the features) who will immediately benefit from it. By contrast, if you have a messy internal function that isn't causing any problems and isn't needed elsewhere, a rewrite might not be justified right now without a clear benefit or consumer.

This approach ensures that refactoring remains **grounded in real-world needs (a “product-market fit” for your code, so to speak)**. Each improved component should slot back into the value chain and deliver tangible value to something above it. Working this way also naturally prioritizes your technical debt backlog: it's fine to leave some rough edges in the code if they're not impacting anyone, and focus on the parts that are actively used and holding you back. By always tying refactoring efforts to concrete demands, you maintain *realism and pragmatism* in development and avoid over-engineering.

Evolutionary Design with LLMs: Using History as the Best Brief

An exciting new dimension to this evolutionary approach is the use of **Large Language Models (LLMs)** (like GPT-4) to assist in coding. One pattern that has emerged is feeding the LLM not just a static spec or request, but the **history of the module's evolution** as part of the prompt. In other words, some of the best “briefs” for coding with an AI include *previous versions of the code or design* that you are trying to improve upon. This is a powerful way to anchor the LLM on the context of the problem.

Consider how this works in practice. Instead of simply telling the LLM, “Write a function that does X,” you can provide a richer prompt: for example, a detailed description of what you need *and* snippets or summaries of earlier attempts (“Here is code from version 1 that was doing X but had issues Y and Z; and here’s code from version 2 which improved on it but still isn’t optimal”). By giving the model this **evolutionary context**, you accomplish several things:

- **The LLM understands the “why” behind the code.** The previous versions and your commentary on them explain the reasoning, constraints, and pitfalls encountered. This guides the AI to produce a solution that addresses known issues and aligns with the intended direction. Essentially, the model gains insight into design decisions and can avoid repeating past mistakes.
- **It reduces guesswork and focuses the AI.** Providing the history means the LLM doesn’t have to infer everything from scratch or sift through an entire large codebase without guidance. You’re focusing its attention on the relevant portions and the changes that matter. The model’s context window is finite, so being selective (not dumping the entire codebase, but just the key evolution snippets) yields better results. The prompt becomes a concise narrative of the module’s journey, which is easier for the LLM to follow than a giant monolithic code dump.
- **It encourages a clean-slate solution, informed by experience.** One of the advantages of this approach is that the LLM is not constrained to just patching the old code; you can explicitly ask it to *rewrite* the component afresh, using the prior version only as a reference. This aligns with the Wardley evolution mindset – when moving from a custom solution to a more productized form, sometimes a rewrite is an opportunity to shed the “legacy” baggage while keeping the learned lessons. The LLM can leverage all the knowledge (e.g., why certain approaches failed, what requirements must be met) to generate a new implementation that is more elegant or efficient. In a sense, you get to **carry forward the wisdom without carrying forward the code’s technical debt**.

Notably, this technique addresses a classic concern in software engineering: *losing knowledge when rewriting code*. Traditionally, rewriting code from scratch has been seen as risky because you throw away years of bug fixes and improvements baked into the old code – as Joel Spolsky famously put it, “*When you throw away code and start from scratch, you are throwing away all that knowledge... Years of programming work.*” ¹⁴ . However, by using an LLM with an evolutionary brief, you are essentially distilling that knowledge into the prompt. The previous code isn’t gone; it’s informing the new code via the AI. This mitigates the risk: the important bug fixes or behaviors from the old version can be mentioned in the brief (“remember to handle the edge-case that we fixed in v2”), so the new solution preserves those insights. In this way, the rewrite becomes more of a guided regeneration.

Moreover, writing these detailed briefs for the LLM has a side benefit for the developers themselves: it forces a **moment of reflection**. You must clearly explain what the component is supposed to do, how it evolved, and why you’re changing it – essentially creating thorough documentation and rationale. This is a best practice with or without AI, but the AI prompt use-case really incentivizes it. It’s similar to

writing a design doc: you clarify your own thinking and often spot improvements or simplifications in the design before even generating the code. When you finally ask the LLM to produce the new code, you are doing so with a much deeper understanding of the problem space, which typically results in better outcomes.

Benefits of an Evolution-Aware Workflow

Adopting this **Wardley-maps-inspired approach** to software development, especially when augmented with LLM assistance, can yield several significant benefits:

- **Faster development through reuse and commoditization:** By continuously refactoring and abstracting modules that prove their value (and relying on existing commodities for generic needs), you create a library of components that dramatically speed up future work. Each piece of code that graduates to a stable “product” or “utility” becomes a platform on which you or your team can build higher-order features more quickly. As Wardley’s research highlights, when a component becomes industrialized, it “*encourages new activities to be built upon [it]*”, leading to a “**rapid rise in novel things**” leveraging that component ¹⁵ ¹⁶. In practical terms, a developer who has a robust set of internal APIs and services (commodity components) can implement new requirements in a fraction of the time, because so much of the needed functionality is already solved and readily available.
- **Higher quality and maintainability:** Code that has been evolved to later stages (product/commodity) tends to be more stable and well-designed, with clearer interfaces and fewer bugs (since it’s been battle-tested and refined). By deliberately moving important code in that direction, you improve the overall **reliability** of the system. Meanwhile, experimental genesis-stage code is kept appropriately isolated and thin, so if it fails or needs to be thrown away, it doesn’t wreak havoc. The system architecture ends up cleaner – with a foundation of solid utilities and a frontier of innovative, but replaceable, experiments.
- **Improved strategic decision-making:** Thinking in terms of Wardley Maps instills a strategic mindset. You constantly ask: *Is this piece of code differentiating for us or is it undifferentiated plumbing? Should we build it, improve it, or can we use an existing solution?* These questions echo the Wardley Map insights used to avoid waste in software development ¹⁷ ¹⁸. It helps justify technical investments (e.g. “We need to spend time refactoring Module X because five different features depend on it – it’s high on the value chain and in a custom stage, so making it a product will pay off”). Conversely, it provides a rationale to push back on certain tasks (“Why are we reinventing this logging framework when it’s essentially a commodity?”).
- **Seamless knowledge transfer and onboarding:** With comprehensive briefs and historical context documented (as you do when preparing LLM prompts or internal design docs), team knowledge about each component’s evolution becomes explicit. New team members can read these narratives to understand why things are the way they are. The Wardley map perspective additionally gives them a mental model of the system’s landscape – which parts are core and novel versus which are standard. This can shorten onboarding time and improve collaboration, since there’s a shared understanding of the “map” of the codebase.
- **Harnessing AI effectively in development:** The practice of feeding LLMs the evolution history of code maximizes the utility of these models. It plays to the strengths of AI (digesting large context and pattern recognition) while mitigating weaknesses (lack of project-specific context). The AI becomes a partner in refactoring and innovation, working from a rich base of knowledge

you provide. This can result in surprisingly high-quality code generation and design suggestions. Essentially, it's a way of doing **"context engineering"** – carefully curating what context the AI sees so that it produces relevant and useful output ¹⁹. As a result, you can achieve certain development tasks much faster, freeing up human developers to focus on higher-level thinking and verification.

Conclusion

The analogy of Wardley Maps in software development offers a powerful lens to guide both architectural strategy and day-to-day coding work. By recognizing the evolutionary stage of each component, developers can make smarter decisions about where to innovate, where to consolidate, and where to leverage existing solutions. This evolutionary approach, when combined with LLM-powered workflows, creates a feedback loop that accelerates learning and coding: past iterations inform new solutions, and new stable solutions pave the way for further innovation.

In essence, **software development is a continuous journey from genesis to commodity, repeated over and over**. Embracing that reality – working on the right things at the right time, and carrying forward lessons learned – is key to maintaining velocity as your codebase grows. As you refactor and improve with purpose, your codebase becomes richer in capabilities (instead of just larger), enabling you to tackle bigger problems with greater speed. And with tools like AI assistants that can absorb the history and context of your projects, you have an unprecedented opportunity to leverage your accumulated knowledge every time you write new code. This means the more code and experience you have, the faster you can go, because each evolutionary step builds a stronger foundation for the next. By mapping out your software's evolution and iterating intelligently, you ensure that you're always *"working on the places of maximum yield"* (as our opening scenario suggested), and ultimately delivering value to users more effectively.

Sources: The concept of Wardley Maps and evolution stages is based on Simon Wardley's strategy mapping framework ¹ ². Definitions of *Genesis*, *Custom-Built*, *Product*, and *Commodity* stages are adapted from Wardley's published materials ⁵ ⁸. The idea that commoditization increases development speed is discussed in Wardley's writings on industrialization in tech ¹². Principles about focusing on user needs and not over-engineering echo Wardley's doctrine and software best practices like YAGNI ⁹ ¹³. Cautionary notes on rewriting code were famously highlighted by Joel Spolsky ¹⁴, though the use of LLMs with historical context offers a modern take on mitigating those risks. All these insights contribute to the approach described in this document.

¹ ⁹ Wardley map - Wikipedia

https://en.wikipedia.org/wiki/Wardley_map

² ¹⁷ ¹⁸ Wardley Maps | Software Architecture Zen

<https://softwarearchitecturezen.blog/2014/09/23/wardley-maps/>

³ ⁴ ⁵ ⁶ ⁷ ⁸ ¹⁰ ¹¹ Evolution Stages - Strategic Terms | Wardley Maps

<https://www.wardleymaps.com/glossary/evolution-stages>

¹² ¹⁵ ¹⁶ Bits or pieces?: Anticipation

<https://blog.gardeviance.org/2016/12/anticipation.html>

¹³ 7 Common Programming Principles That Every Developer Must Follow

<https://www.geeksforgeeks.org/blogs/7-common-programming-principles-that-every-developer-must-follow/>

14 Things You Should Never Do, Part I – Joel on Software

<https://www.joelonsoftware.com/2000/04/06/things-you-should-never-do-part-i/>

19 How can I provide a large amount of context to an LLM? - Reddit

https://www.reddit.com/r/learnmachinelearning/comments/1fitoin/how_can_i_provide_a_large_amount_of_context_to_an/